

DOM-Graph RAG: DOM-Guided Adaptive Tiling and Budgeted Online Evidence Control for Multimodal Web-Page RAG

Anonymous ACL submission

Abstract

Recent visual retrieval-augmented generation (RAG) systems render documents as pixel screenshots and let a vision-language model (VLM) read tiles of the image directly, avoiding the information loss of OCR and text extraction. PixelRAG (Wang et al., 2026) popularized this idea but tiles the page on a fixed pixel grid, blind to content boundaries, so a table or paragraph can be split mid-tile. Separately, MAGE-RAG (Zuo et al., 2026) organizes long documents into a multigranular evidence graph and reads it with an online controller that activates, opens, and prunes graph nodes under a fixed budget, but targets already-parsed PDFs rather than rendered web pages. We present **DOM-Graph RAG**, which replaces PixelRAG’s blind grid with tiling guided by the page’s own DOM structure, so tile boundaries follow content boundaries by construction. The resulting tiles form a MAGE-RAG-style multigranular graph, extended across pages via hyperlink edges into a small corpus graph. We fine-tune a VLM reader contrastively on synthetic (tile, question) pairs with mined hard negatives (LoRA, InfoNCE), and implement an online evidence controller that evolves every node through a four-state machine (INACTIVE→ACTIVE→OPENED/PRUNED) via best-first search under budget, giving the reader a small, prioritized evidence subgraph instead of the whole page.

We describe the full, open-sourced pipeline and report three preliminary results. First, on a slice of our own synthetic QA set, DOM-guided tiling’s lexical retrieval signal is $15.7\text{--}15.8\times$ stronger than chance, against $3.9\times$ for a blind fixed-height grid at the same resolution, once pool-size differences between the two tilings are controlled for. Second, on a held-out article-level validation split, the fine-tuned reader nearly doubles Recall@1 over the base model ($0.433 \rightarrow 0.785$, MRR $0.578 \rightarrow 0.856$) at realistic per-page retrieval

scale. Third, on cross-page multi-hop questions built from HotpotQA, the controller’s current `reading_order`-only frontier expansion never beats a static top- k baseline over the same scores, turning a limitation already flagged qualitatively into a measured one. Full benchmark evaluation against MAGE-RAG’s published LongDocURL and MMLongBench-Doc numbers is ongoing work, reported as such in Section 5.

1 Introduction

Retrieval-augmented generation (RAG) over visually rich documents – web pages, tables, infoboxes – has long relied on text extraction: parse the HTML or run OCR, chunk the text, retrieve with a dense retriever. This discards exactly what layout encodes: which numbers belong to which table row, which caption belongs to which figure. PixelRAG (Wang et al., 2026) instead shows that a VLM retrieving directly over rendered page screenshots can match or beat text-based retrieval on long, layout-heavy documents – but its tiling is a fixed pixel grid, blind to page content: every tile has the same height regardless of content, so a table row or paragraph can be split with no awareness it was ever one unit. A retriever built on such tiles must reconstruct boundaries the renderer already knew and threw away.

MAGE-RAG (Zuo et al., 2026) addresses a different problem: given an already-segmented long document, which passages should an agent read under a bounded budget? It builds a multigranular evidence graph (containment, reading order, layout adjacency, section hierarchy, semantic neighbors) and an online controller that activates, opens, and prunes nodes under budget – but assumes the document is already parsed, and does not address obtaining content-aligned tiles from a rendered page in the first place.

The two are complementary: PixelRAG shows pixel tiles read by a VLM are a viable retrieval

unit, but tiles blindly; MAGE-RAG shows a budget-aware graph controller beats static top- k , but assumes elements already exist. This paper combines them via the one artifact PixelRAG discards and MAGE-RAG never had to construct: the DOM. For a rendered web page, the browser’s own layout engine already knows every element’s exact bounding box before a single pixel is cropped.

Contributions. We introduce **DOM-Graph RAG**, an open-source pipeline¹ (§3) that:

1. replaces PixelRAG’s blind grid with *DOM-guided adaptive tiling*: elements cropped along their own bounding boxes, undersized ones merged, oversized ones split, headings paired with their content (§3.1);
2. organizes tiles into a *multigranular graph* in the style of MAGE-RAG, instantiating all five of its relations plus a new cross-page hyperlink relation and a *continues* relation for oversized pages split into sections (§3.2);
3. fine-tunes a general-purpose VLM into a page-element reader via *contrastive LoRA training* on synthetic (tile, question) pairs with mined hard negatives, mirrored at reduced resolution to test PixelRAG’s resolution/token-cost claim (§3.3);
4. implements an *online evidence controller* generalizing MAGE-RAG’s controller to live-DOM graphs via a four-state machine under a node-opening budget (§3.4).

The full pipeline is implemented and exercised end-to-end on live Wikipedia pages, with dependency-light unit tests over the geometric, graph, mining, and image-processing logic. §5 reports three preliminary ablations and lays out the evaluation against MAGE-RAG’s published baselines as work in progress, rather than fabricate numbers ahead of running it.

2 Related Work

Text-based document RAG. Standard RAG chunks extracted text and retrieves with a dense bi-encoder (Karpukhin et al., 2020). On visually rich documents this requires a layout-aware parser or OCR, both lossy on complex layouts, and discards 2D structure before retrieval ever runs.

Screenshot-based visual RAG. PixelRAG (Wang et al., 2026) retrieves over rendered-page image tiles with a VLM embedding model, part of an established line retrieving over page images rather than text: ColPali (Faysse et al., 2024) (late-interaction patch embeddings), Document Screenshot Embedding (Ma et al., 2024a) (whole-page VLM embeddings, no parsing), and VisRAG (Yu et al., 2024)/M3DocRAG (Cho et al., 2024) (retrieve-then-generate pipelines, the latter multi-page). None address how a page is cut into retrievable units – what our tiling (§3.1) targets, so it could feed any of them, not only PixelRAG, whose own blind fixed-height grid lets a table or paragraph straddle a tile boundary. PixelRAG also reports up to $3\times$ token-cost reduction at lower resolution without specifying the filter or resolutions tested; we build a matched full-/reduced-resolution pair on our own tiles to measure this directly (§3.3).

Graph-structured evidence and controllers. MAGE-RAG (Zuo et al., 2026) (evaluated on LongDocURL (Deng et al., 2025), MMLongBench-Doc (Ma et al., 2024b)) builds an offline multigranular evidence graph and reads it with a budgeted controller – the direct ancestor of §3.4. The difference is provenance: we build the graph from a rendered page’s DOM, giving exact boundaries and reading order for free, and instantiate all five of its relation types (§3.2). GraphRAG (Edge et al., 2024) shares the premise of building an explicit graph before querying it, over extracted entities rather than layout structure. Layout- and markup-aware pretraining (LayoutLM (Xu et al., 2020), MarkupLM (Li et al., 2022)) instead bakes structure into the model’s representation; we use the DOM directly at inference time, with no pretraining required. Web-agent work (WebArena (Zhou et al., 2023), Mind2Web (Deng et al., 2023), SeeAct (Zheng et al., 2024)) has already studied when DOM structure is reliable enough to act on – the same failure mode (JS-hydrated, div-soup markup) that scopes our own claim to semantically marked-up pages (§6).

Contrastive fine-tuning and hard-negative mining. Training with in-batch negatives plus mined hard negatives is standard in dense retrieval, e.g. DPR’s BM25-mined negatives (Karpukhin et al., 2020); we use TF-IDF at tile granularity instead. Training pairs are generated synthetically

¹Anonymized for review: [TODO: paste anonymous.4open.science link here before submitting]

by prompting a VLM per tile, in the spirit of InPars (Bonifacio et al., 2022) and Promptagator (Dai et al., 2022) adapted to image tiles. We fine-tune with LoRA (Hu et al., 2021) on both the decoder and the vision tower.

3 Method

A page is rendered once with a headless browser (Appendix E, Figure 2 diagrams the flow); its DOM geometry drives an adaptive tiling step (§3.1); tiles are organized into a two-level graph, optionally spanning several hyperlinked pages (§3.2); a VLM reader is contrastively fine-tuned to score (question, tile) pairs (§3.3); and, at query time, an online controller walks the graph under a budget to decide which nodes the reader actually opens (§3.4).

3.1 DOM-Guided Adaptive Tiling

Rendering and extraction. We load the page in headless Chromium at a fixed viewport width (2048px), remove interface chrome via a CSS selector list, and take a full-page screenshot. For a configurable tag set (h1–h4, p, table, figure, img, ul/ol, blockquote, pre) we read each element’s absolute bounding box from the rendered DOM. For non-text containers this is `getBoundingClientRect`; for text-flow tags, whose CSS box spans the full container width regardless of actual content, we instead union the client rects of every descendant text node. Without this, a paragraph or reference list sharing a row with a floated infobox reports a bounding box running through the infobox’s column even though its text stops well short of it. Hyperlinks are collected from `<p>` elements only, seeding the cross-page edges of §3.2.

Overlap resolution. Nested tags (an `<img>` in a `<figure>`) yield duplicate bounding boxes for the same content; we sort by DOM depth and drop any element already fully contained (1px tolerance) in a shallower kept element. A floated sidebar and the text sharing its row can also genuinely intersect even after the text-flow tightening above. Rather than tile such elements independently and duplicate the shared pixels, we connect any two elements whose bounding-box intersection exceeds a small fraction of the smaller one’s area, and tile each connected component as one crop spanning their union.

Adaptive tiling and patching. Elements are processed top-to-bottom under three rules: (1) *heading pairing* – a heading is never a standalone tile, but merged with the next element (e.g. h2+p); (2) *merging* – non-heading elements under 80px are buffered until their union spans that minimum, then flushed as one merged tile; (3) *patching* – an element taller than 2048px is sliced into $\lceil h/h_{\max} \rceil$ equal-height sub-tiles. This targets PixelRAG’s failure mode directly: a table that would straddle a fixed grid cell is instead one tile, and a short caption merges with its neighbor rather than being orphaned. Each tile keeps a pointer to its source element(s), used both for the graph (§3.2) and as ground truth for negative-mining (§3.3). Figure 3 (Appendix E) shows tiles overlaid on a real page.

3.2 Multigranular Graph Construction

Tiles are organized into a two-level directed graph (one page node, one element node per tile) with all five of MAGE-RAG’s relation types, plus one of our own for cross-page connectivity:

- **contains** (page $\rightarrow$ element): every element belongs to its page.
- **reading_order** ($\text{element}_i \rightarrow \text{element}_{i+1}$): a spatial heuristic, *not* DOM order – elements are grouped into visual rows (20px vertical-overlap tolerance), each sorted left-to-right, rows concatenated top-to-bottom, since DOM and visual order diverge under CSS positioning.
- **layout_adjacency** (element $\leftrightarrow$ element): reciprocal edges to the 2D neighbors reading order flattens away (left/right within a row; above/below to the nearest horizontally-overlapping tile in the next row).
- **section_hierarchy** (heading $\rightarrow$ child): each heading points to its nested sub-heading/content, forming a section tree distinct from the flat `contains` relation.
- **semantic_neighbor** (element $\leftrightarrow$ element): TF-IDF-cosine edges between lexically close elements regardless of position, added greedily under a per-tile degree cap. Optional, off by default, meant to be toggled to compare the controller with and without it (§5).
- **links_to** (element $\rightarrow$ external_page): hyperlinks from the first $k=3$ link-bearing

text tiles (reading order); beyond that budget, links are dropped to bound the corpus crawl’s branching factor.

Every element node starts with a `state` attribute (INACTIVE), which the controller (§3.4) evolves at query time. Figure 4 (Appendix E) shows the resulting graph for the page of Figure 3.

Oversized-page splitting. One article’s screenshot exceeded 46,000 px and produced dozens of tiles whose sequential QA generation (§3.3) ran for hours with no partial progress saved on failure. Rather than exclude such pages, one whose element height exceeds $10\times$ the max tile height is split into sections *before* tiling, at `h1/h2` boundaries, with any still-oversized section further rechunked by a height budget. Each section is tiled and graphed as a standalone page would be; the per-section graphs are merged by node-namespacing, with consecutive sections’ page nodes linked by a new **continues** relation – distinct from `links_to`: sequential parts of the same document, not a hyperlink elsewhere. Every page’s graph, split or not, exposes an *entry point* for the corpus graph below to rewire onto; the controller does not yet cross `continues` edges automatically (§6). Figure 5 (Appendix E) shows an example split into 16 sections.

Corpus graph. From a seed page we crawl up to m pages reachable via its `links_to` edges (one hop, non-recursive), build each linked page’s graph independently, and merge by namespacing every node with a URL-derived slug, rewiring each placeholder external-page node onto the crawled page’s entry point. `links_to` is the only cross-document connective tissue, `continues` the only cross-section one – the substrate the evidence controller (§3.4) operates over.

Structured rendering. Any (sub-)graph serializes its page node and directly-contained elements to XML (one `<element id=... tag=... state=...>text</element>` per node) as the reader’s structured input, in the spirit of MAGE-RAG’s structured-rendering step (example in Appendix E, Figure 6).

3.3 Contrastive Reader Fine-Tuning

To turn a general-purpose VLM into a tile-level reader that also scores how well a tile answers a query, we fine-tune it contrastively on synthetic data, in three stages.

Synthetic QA generation. For every tile, we prompt the VLM with the tile image alone for one self-contained, factual question answerable *only* from that tile, or `SKIP` if it carries no exploitable content – the recipe of synthetic query generation for retriever training (InPars, Promptagator; Bonifacio et al., 2022; Dai et al., 2022) applied to image tiles. Every tile crop is persisted regardless of outcome, so `SKIPPED` tiles remain available as candidate hard negatives.

Hard-negative mining. For each question, we rank every other tile from the same page by TF-IDF cosine similarity and keep the top $n=2$ clearing two false-negative filters: (a) a lexical filter rejecting any tile whose text already contains the answer verbatim, and (b), optionally, a VLM-judge filter showing the candidate image to the reader VLM and asking whether it can recover the answer there, catching non-lexical false negatives at the cost of one extra VLM call per candidate.

Contrastive LoRA fine-tuning. Qwen2-VL is generative, not a similarity encoder; we turn it into one via PromptEOL (Jiang et al., 2024) (a short fixed instruction, “summarize the above in one word,” taking the last token’s final-layer hidden state as embedding), originally proposed for text-only LLMs and applied here to a VLM’s image+instruction input. LoRA ($r=16$, $\alpha=32$, dropout 0.05) adapts both the decoder’s attention projections and the vision tower’s, so the model can refine what it attends to in the image jointly with how it reads the question. Given B questions each with one positive and K mined negatives, we minimize a temperature-scaled ($\tau=0.05$) InfoNCE loss over the $B(1+K)$ tiles in the batch, so each positive competes against its own negatives and, in-batch, every other question’s tiles.

Resolution as an efficiency lever. PixelRAG reports up to $3\times$ token-cost reduction at lower resolution without specifying the filter or resolutions tested. We build the infrastructure to test this directly: every tile can be mirrored via Lanczos resampling at a configurable scale (0.5 default), reusing the same questions/positives/negatives at both resolutions so comparing them is a matter of pointing fine-tuning at one image root or the other (§5).

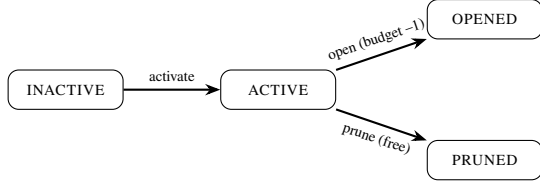


Figure 1: The controller’s state machine. Opening a node consumes budget and may activate inactive reading_order neighbors; pruning is free, applied to any node still ACTIVE once budget is exhausted.

3.4 Online Evidence Controller

Given a query, the controller decides which element nodes the reader actually reads, rather than the whole page or a static top- k . It evolves each node’s state through the four-state machine of Figure 1, generalizing MAGE-RAG’s Algorithm 1.

Relevance scoring. Every element node is scored against the query by TF-IDF cosine similarity (same mechanism as hard-negative mining), a deliberate placeholder for the fine-tuned reader’s own learned similarity (§6).

Policy. Algorithm 1: (1) seed the frontier with the k highest-scoring inactive nodes; (2) repeatedly take the highest-scoring *active* node, pruning it for free below an opening threshold or opening it (one unit of budget B) and activating still-inactive reading_order neighbors that clear an activation threshold; (3) once budget is exhausted or no active node remains, prune whatever is left active. Neighbors are only considered *after* their node is opened, so the frontier grows along reading order conditioned on what has already proven relevant, rather than a fixed global ranking – what distinguishes it from static top- k retrieval over the same scores (ablation (d) in §5).

Output. The concatenation of OPENED nodes’ text, in reading order, is the final evidence passed to the downstream reader, with the controller’s action log available for inspection.

4 Implementation

The pipeline is implemented in Python, one module per stage of §3: rendering/extraction via Playwright, DOM-guided tiling over Pillow crops, graph construction with networkx, a cached VLM client (Ollama’s qwen3-v1 by default, or Hugging Face Qwen2-VL-7B-Instruct for

Algorithm 1 Budgeted best-first evidence control

Require: graph G , query q , budget B , seed count k , thresholds $\theta_{\text{open}}, \theta_{\text{act}}$

- 1: score every element node by $\text{sim}(q, \text{text}(n))$
- 2: activate the k highest-scoring inactive nodes $\triangleright$ seed
- 3: $b \leftarrow 0$
- 4: **while** $b < B$ and some node is ACTIVE **do**
- 5: $n \leftarrow \arg \max$ score over ACTIVE nodes
- 6: **if** $\text{score}(n) < \theta_{\text{open}}$ **then**
- 7: $n \leftarrow \text{PRUNED}$ $\triangleright$ free
- 8: **else**
- 9: $n \leftarrow \text{OPENED}$; $b \leftarrow b + 1$; record its text as evidence
- 10: **for each** inactive reading_order neighbor n' of n **do**
- 11: **if** $\text{score}(n') \geq \theta_{\text{act}}$ **then**
- 12: $n' \leftarrow \text{ACTIVE}$
- 13: **end if**
- 14: **end for**
- 15: **end if**
- 16: **end while**
- 17: prune every node still ACTIVE $\triangleright$ finalize
- 18: **return** concatenation of OPENED nodes’ text, in reading order

LoRA fine-tuning), synthetic QA generation, TF-IDF hard-negative mining, and contrastive LoRA fine-tuning with peft/torch. Oversized-page splitting (§3.2) is a pure function unit-testable without a browser; page fetching, tiling, and graph composition are checkpointed per page unit so a failure partway through a long page only costs the section in flight. Dependency-light unit tests cover the geometric, graph, mining, and image-processing logic on synthetic fixtures; Playwright-backed tests check DOM extraction end-to-end against a bundled HTML fixture. Appendix A gives the full module-level breakdown.

5 Evaluation Protocol and Status

The pipeline of §3 runs end-to-end on live pages and the trained reader is now evaluated held-out, but the full benchmark suite against MAGE-RAG’s own baselines is not yet in place. We report three preliminary ablations below.

Benchmarks. MAGE-RAG’s protocol evaluates on LongDocURL (Deng et al., 2025) and MMLongBench-Doc (Ma et al., 2024b) (Table 1), but both are PDF benchmarks, and every input our method needs (DOM bounding boxes, CSS-

Benchmark	Metric	MAGE-RAG
LongDocURL	Accuracy	52.75
MMLongBench-Doc	Acc. / F1	53.26 / 51.19

Table 1: Published MAGE-RAG (Zuo et al., 2026) numbers, the reference our controller (§3.4) generalizes. Our results on the same benchmarks are ongoing work.

selector chrome removal, h1/h2 splitting) presupposes a renderable DOM a PDF does not have. We instead plan an HTML-native source: WebSRC (Chen et al., 2021) or HotpotQA (Yang et al., 2018), whose `supporting_facts` name the exact sentences, in two articles, needed per question – multi-hop by construction and the most direct fix for our own synthetic QA’s single-hop bias (§6). We have implemented and scaled the HotpotQA path (`src/multihop_qa.py`, §6): 91/150 sampled questions (61%) yielded a complete cross-page example, used directly in ablation (d) below. We do not evaluate on SimpleQA: with no supplied document, it cannot exercise tiling, the graph, or the controller.

Ablation (a): DOM-guided tiling vs. a blind fixed-height grid. The lexical scorer already used for hard-negative mining and controller relevance (§3.3, §3.4) lets us run this ablation without waiting on the trained reader. `scripts/grid_baseline.py` reimplements a PixelRAG-style blind grid (1,024px bands, PixelRAG’s own reported tile height, at our render width); `scripts/dom_vs_grid_experiment.py` re-renders 18 single-unit pages, tiles each both ways, and ranks every existing `qa_pairs.jsonl` question against both tile sets by TF-IDF cosine – same scorer, only the tiling differs. A question is scored in a condition only if its answer string occurs in one of that condition’s tiles, else dropped from that condition rather than counted as a miss.

Raw numbers say grid is more accurate – the opposite of this paper’s claim – but are confounded: grid’s pool is $\sim 5\times$ smaller (Appendix D), which improves Recall/MRR mechanically regardless of ranking quality. Normalizing by each row’s own chance-level MRR reverses the conclusion: DOM-guided tiling is **15.7–15.8 $\times$** better than chance on its own pool, against grid’s **3.9 $\times$** on its own – a gap that survives on the paired subset too, so it is not

	n	R@1	MRR	MRR/chance
<i>All attributable pairs</i>				
DOM	212	0.547	0.663	15.8 $\times$
Grid	108	0.713	0.811	3.9 $\times$
<i>Paired (107 pairs attributable in both)</i>				
DOM	107	0.505	0.659	15.7 $\times$
Grid	107	0.720	0.817	3.9 $\times$

Table 2: DOM-guided vs. blind grid, same TF-IDF scorer, 18 pages / 409 questions. $\text{MRR/chance} = \text{MRR} \div \text{mean chance-level MRR } (1/n_{\text{items}})$ of that row’s own pool, since grid’s pool is $\sim 5\times$ smaller (Appendix D) and inflates raw Recall/MRR mechanically.

	R@1	R@3	R@5	MRR
Base (no LoRA)	0.433	0.645	0.752	0.578
LoRA fine-tuned	0.785	0.915	0.952	0.856

Table 3: Base vs. LoRA-fine-tuned reader, held-out val split (330 questions, article-level, never seen in training; 8 dropped: page with <2 tiles or positive tile absent from the manifest).

an artifact of differing question sets. We read this as a real retrieval advantage from content-aligned boundaries, though a preliminary signal (single lexical scorer, self-generated single-hop questions, §6) rather than a final result.

Ablation (b): fine-tuned reader vs. base model, held-out retrieval. We added an article-level train/val split (`split_examples_by_article`, 80/20, disjoint articles in each split so no tile or near-duplicate question leaks across) and retrained LoRA on the train split only, on rented GPU compute (3 epochs, batch size 4, gradient checkpointing). `scripts/eval_retrieval.py` then scores every val-split question against every tile of its own page (a realistic retrieval pool, not the small mined positive/negatives set the training loss sees), with and without the adapter loaded on the same base model (`peft`’s `disable_adapter()`), so both passes share one model in memory).

Recall@1 nearly doubles (0.433 $\rightarrow$ 0.785) and MRR rises from 0.578 to 0.856 – a clean, held-out signal that the contrastive objective transfers to realistic per-page retrieval, not just the small mined pool it was trained against. This motivates wiring this reader into the controller’s relevance scoring in place of TF-IDF (§6), left as the next step.

Seed width k_{seed}	Controller	Static top- k
3 (default; budget 5)	29.7% (27/91)	40.7% (37/91)
5 (= budget)	40.7% (37/91)	40.7% (37/91)

Table 4: Fraction of 91 cross-page multi-hop questions (§6) where the selected evidence covers both cited articles’ positive tiles. Same TF-IDF scores in every cell; only the selection policy differs, and, between rows, whether any budget remains for propagation past the seed.

Ablation (d): evidence controller vs. static top- k , cross-page multi-hop retrieval. Method (§3.4) motivates the controller against the static top- k baseline it generalizes – untestable on single-hop synthetic QA, where one tile always suffices, so we use the 91 HotpotQA-derived multi-hop questions of §6 instead. For each, we build the combined two-page graph of both cited articles, run the controller and, on the identical TF-IDF scores, a static top- k baseline, and check whether the evidence covers a positive tile on *each* page.

At the controller’s default $k_{\text{seed}}=3$ it is *worse* than the baseline; at $k_{\text{seed}}=5=\text{budget}$ the two are exactly equal, since the controller then opens its seed outright with no budget left to propagate. Both rows share one cause: `reading_order` never crosses a page boundary, so the controller can only reach a second page via its literal top- k_{seed} seed, exactly what static top- k already does. This is a negative result for the current controller, not graph-guided retrieval in general: it turns §6’s qualitative claim into a measured one, and marks type-aware expansion as necessary, not merely desirable, for multi-hop tasks.

Status. All stages (§3) are implemented, tested, and demonstrated end-to-end on live pages (§4). QA-pair generation has scaled to the multi-page corpus of Table 5. Scaling surfaced one cost worth flagging: ~ 2 minutes/tile for local QA generation, now partly mitigated by concurrent VLM calls (§4); Appendix C turns this into a compute-budget estimate for fine-tuning itself. Ablation (c) (resolution) still requires a further training run on compressed tiles.

6 Limitations

Relation coverage is complete, consumption is not. All five of MAGE-RAG’s relation types are instantiated (§3.2), but the controller’s frontier expansion (Algorithm 1) only ever follows

Statistic	Value
Distinct Wikipedia articles	19
Page units (articles + split sections)	185
Tiles extracted	1,784
QA pairs generated	1,717
Tiles with no usable QA (SKIP)	67 (3.8%)
Contrastive examples (QA + hard negatives)	1,339

Table 5: Contrastive dataset size (§3.3), lexical filter only (no VLM judge). 12 of 19 articles were split into sections (§3.2).

`reading_order:` `layout_adjacency`, `section_hierarchy`, and `continues` are built and visualized but never queried, and `semantic_neighbor` is off by default. Operationally the graph the controller walks is closer to a chain than the structure it is built from. Type-aware, per-relation expansion is the natural next step: ablation (d) (§5) shows the controller never beats, and at its default seed width underperforms, a static top- k baseline over the same scores on cross-page multi-hop questions.

Single-hop self-generated data cannot exercise the graph – addressed via HotpotQA. §3.3’s QA generation asks for a question answerable only from the tile shown, so an oracle controller could answer every such question from one node: the graph cannot be shown to help on this data. `src/multihop_qa.py` closes this gap with HotpotQA (Yang et al., 2018) instead: each question’s `supporting_facts` names two Wikipedia articles and the sentences in each needed to answer it, so we render both, keep tiles overlapping a supporting sentence as positives, and mine hard negatives by TF-IDF pooled across both articles’ tiles. Scaled to 91/150 sampled questions (61%) and used directly in ablation (d) (§5), this shows the controller currently underusing the graph it is given. Using this data for training, rather than evaluation only, remains future work.

Scoped to semantically marked-up pages. Chrome removal uses a MediaWiki-specific selector list, and every result here is on Wikipedia; we do not claim this works unmodified on JS-hydrated SPAs or `div-soup` layouts, the regime web-agent work such as WebArena (Zhou et al., 2023), Mind2Web (Deng et al., 2023), and See-Act (Zheng et al., 2024) already studies from the acting side. The open web is future work.

Same-model-family generation, judging, and reading. Synthetic questions, the optional false-negative judge, and the fine-tuned reader all come from the qwen3-vl/Qwen2-VL family, so any measured gain could partly reflect the generator’s idiosyncrasies rather than genuine document understanding; hand-verifying generated pairs, and eventually human-authored questions, is needed to rule this out.

Arbitrary slicing, lexical scoring, and other gaps. Patching (§3.1) splits an oversized element into equal-height sub-tiles regardless of content, a narrower version of PixelRAG’s own failure mode; a dynamic, resolution-preserving scheme (Qwen2-VL’s AnyRes) is the natural fix. The controller’s TF-IDF relevance scoring (§3.4) misses paraphrases and cross-modal cues the fine-tuned reader would catch; wiring in the reader’s own similarity is the natural next iteration. The VLM-judge false-negative filter is opt-in and unused so far; hard-negative mining is single-page outside the HotpotQA path above; the corpus crawl is one hop, non-recursive; and all hyperparameters (Appendix B) are chosen once, not swept.

7 Conclusion

DOM-Graph RAG grounds two ideas in visual and graph-structured RAG – PixelRAG’s screenshot tiles and MAGE-RAG’s budgeted graph controller – in the DOM a rendered web page already provides for free: content-aligned tiling replaces a blind pixel grid, a multigranular graph replaces a flat tile list, a contrastively fine-tuned reader replaces a generic VLM, and a four-state evidence controller replaces reading the whole page or a static top- k . Every stage runs end-to-end on live pages. Three preliminary results (§5) support this design: DOM-guided tiling carries a real retrieval advantage over a blind grid once pool-size effects are controlled for; the fine-tuned reader substantially outperforms the base model at realistic per-page retrieval; and, on genuinely multi-hop questions, the controller’s current `reading_order`-only expansion cannot yet beat a static top- k baseline – a measured version of a limitation §6 already flagged, not a new one. Next: an HTML-native benchmark, the resolution ablation, and closing the gaps of §6 – most importantly, type-aware frontier expansion beyond `reading_order` and wiring the controller’s relevance scoring to the now-validated reader in place of the TF-IDF place-

holder.

Ethics Statement

The corpus builder fetches a seed Wikipedia page plus a small, bounded number of linked pages, with retry-and-backoff rather than aggressive re-fetching; we crawled only Wikipedia, built for programmatic access at this volume, though a production deployment should prefer the Wikimedia REST API or a static dump over live scraping, which this prototype does not do by default. Wikipedia text is CC BY-SA; embedded images carry heterogeneous, some non-free, licenses, so the screenshots/tiles produced are derivative works we do not release here (a future release would need per-image licensing). Synthetic questions (§3.3) are VLM-produced and unverified, and should be labeled as such if released (§6). This work involves no human subjects or user study.

References

- Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. [Inpars: Data augmentation for information retrieval using large language models](#). *Preprint*, arXiv:2202.05144.
- Xingyu Chen, Zihan Zhao, Lu Chen, Danyang Zhang, Jiabao Ji, Ao Luo, Yuxuan Xiong, and Kai Yu. 2021. [WebSRC: A dataset for web-based structural reading comprehension](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Jaemin Cho, Debanjan Mahata, Ozan Irsoy, Yujie He, and Mohit Bansal. 2024. [M3docrag: Multi-modal retrieval is what you need for multi-page multi-document understanding](#). *Preprint*, arXiv:2411.04952.
- Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. 2022. [Promptagator: Few-shot dense retrieval from 8 examples](#). *Preprint*, arXiv:2209.11755.
- Chao Deng, Jiale Yuan, Pi Bu, Peijie Wang, Zhongzhi Li, Jian Xu, Xiao-Hui Li, Yuan Gao, Jun Song, Bo Zheng, and Cheng-Lin Liu. 2025. [Long-DocURL: a comprehensive multimodal long document benchmark integrating understanding, reasoning, and locating](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1135–1159, Vienna, Austria. Association for Computational Linguistics.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su.

688	2023. Mind2web: Towards a generalist agent for the web . <i>Preprint</i> , arXiv:2306.06070. NeurIPS 2023.	<i>SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)</i> .	744
689			745
690	Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization . <i>Preprint</i> , arXiv:2404.16130. Microsoft.	Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering . In <i>Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)</i> .	746
691			747
692			748
693			749
694			750
695	Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. 2024. Colpali: Efficient document retrieval with vision language models . <i>Preprint</i> , arXiv:2407.01449.		751
696			752
697			
698			753
699			754
700	Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models . <i>Preprint</i> , arXiv:2106.09685.	Shi Yu, Chaoyue Tang, Bokai Xu, Junbo Cui, Junhao Ran, Yukun Yan, Zhenghao Liu, Shuo Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2024. Visrag: Vision-based retrieval-augmented generation on multi-modality documents . <i>Preprint</i> , arXiv:2410.10594.	755
701			756
702			757
703			758
704			
705	Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2024. Scaling sentence embeddings with large language models . In <i>Findings of the Association for Computational Linguistics: EMNLP 2024</i> , pages 3182–3196.	Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a generalist web agent, if grounded . <i>Preprint</i> , arXiv:2401.01614. ICML 2024.	759
706			760
707			761
708			762
709			
710	Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In <i>Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)</i> .	Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. Webarena: A realistic web environment for building autonomous agents . <i>Preprint</i> , arXiv:2307.13854. ICLR 2024.	763
711			764
712			765
713			766
714			767
715			768
716	Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. 2022. MarkupLM: Pre-training of text and markup language for visually-rich document understanding . In <i>Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)</i> .	Yilong Zuo, Xunkai Li, Jing Yuan, Qiangqiang Dai, Hongchao Qin, and Ronghua Li. 2026. MAGE-RAG: Multigranular adaptive graph evidence for agentic multimodal RAG in long-document QA . <i>Preprint</i> , arXiv:2606.15906. Code: github.com/laonuo2004/MAGE-RAG .	769
717			770
718			771
719			772
720			773
721			774
722	Xueguang Ma, Sheng-Chieh Lin, Minghan Li, Wenhu Chen, and Jimmy Lin. 2024a. Unifying multimodal retrieval via document screenshot embedding . In <i>Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)</i> .		
723			
724			
725			
726	Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yungang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. 2024b. Mmlongbench-doc: Benchmarking long-context document understanding with visualizations . <i>Preprint</i> , arXiv:2407.01523.		
727			
728			
729			
730			
731			
732			
733	Yichuan Wang, Zhifei Li, Zirui Wang, Paul Teilletche, Lesheng Jin, Matei Zaharia, Joseph E. Gonzalez, and Sewon Min. 2026. Pixelrag: Web screenshots beat text for retrieval-augmented generation . <i>Preprint</i> , arXiv:2606.28344. UC Berkeley / BAIR / Berkeley NLP. Code: github.com/StarTrail-org/PixelRAG .		
734			
735			
736			
737			
738			
739			
740	Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. LayoutLM: Pre-training of text and layout for document image understanding. In <i>Proceedings of the 26th ACM</i>	A Implementation Details	775
741		The pipeline is Python, one module per stage	776
742		of §3: rendering/extraction via Playwright	777
743		(dom_extraction.py); tiling and pruning	778
		over Pillow crops (tiling.py); graph construc-	779
		tion with networkx (graph_builder.py);	780
		the reader VLM via Ollama (qwen3-vl,	781
		default, no GPU needed) or Hugging Face	782
		transformers (Qwen2-VL-7B-Instruct)	783
		for LoRA fine-tuning, every call cached to disk by	784
		a hash of (image, prompt, decoding-config) since	785
		decoding is greedy, so a retrieved page never re-pays	786
		a call already made (vlm_client.py); syn-	787
		thetic QA generation (qa_generation.py);	788
		hard-negative mining via scikit-learn	789
		TF-IDF with a dependency-free fallback	790
		(hard_negative_mining.py); and con-	791
		trastive LoRA fine-tuning with peft/torch,	792
		target modules discovered by introspecting	793
		the loaded model rather than hard-coded	794
		(lora_finetune.py).	795

Oversized-page splitting, composition, and concurrency. `sectioning.py` implements the h1/h2 split as a pure function over already-extracted elements, unit-testable without Playwright. `corpus_builder.py` separates network-bound fetch from a synchronous core that tiles, decides whether to split, and composes per-section graphs, exposing an `entry_page` on every graph; multi-page (`links_to`) and multi-section (`continues`) composition share the same node-namespacing machinery. QA generation issues per-tile VLM calls concurrently within a page (a thread pool sized to Ollama’s default parallelism); the controller does the same on the query path, since which nodes open depends only on relevance scores computed upfront, never on a VLM response.

Batch construction and resolution ablation. `scripts/build_dataset.py` drives the pipeline over a page list with retrying fetches; its unit of work is a *page unit* (a page, or one split section), checkpointed independently to `contrastive_examples.jsonl` – a failure partway through an oversized page costs only the section in flight, motivated by an early run that lost hours of work when the whole page was the retry unit. `image_compression.py` mirrors tile images at a reduced scale under a separate root with identical filenames and relative paths, so switching a training run between resolutions (§3.3) is a single `--images-root` flag.

Tiling ablation and multi-hop data. `scripts/grid_baseline.py` reimplements the blind grid as a module independent of `tiling.py`; `scripts/dom_vs_grid_experiment.py` re-renders each page, builds both tilings, and ranks existing questions against both by TF-IDF (§5). `src/multihop_qa.py` builds cross-page examples from HotpotQA (Yang et al., 2018) records streamed via the `datasets` library; `scripts/build_multihop_dataset.py` reports unreachable-article vs. unmatched-sentence drops separately. Positive-tile attribution uses stopword-filtered token overlap rather than exact substring matching, since the latter rarely survives a decade of article edits since HotpotQA’s 2017 source dump.

Demonstration and tests. A notebook runs every stage end-to-end with interactive `pyvis`

graph visualizations. Dependency-light unit tests cover the geometric pipeline, graph construction and splitting, hard-negative mining (lexical and VLM-judge, stub client), the VLM cache and call concurrency, and resolution compression, all on synthetic fixtures needing no network, VLM, or GPU. Playwright-backed tests check the text-flow bounding-box measurement (floated-sidebar fixture) and the full geometric pipeline end-to-end (DOM extraction through XML export) against a bundled local HTML fixture.

B Hyperparameters

Parameter	Value
Viewport / max tile width	2048 px
Max tile height (patching)	2048 px
Min tile height (merging)	80 px
Row-grouping tolerance	20 px
Nesting tolerance	1 px
Overlap-grouping threshold	15% of smaller area
Link-bearing tiles kept (k)	3
Oversize-split threshold	$10 \times$ max tile height
Mined hard negatives (n)	2
Semantic-neighbor threshold	0.2 cosine
Semantic neighbors / tile (max)	3
Downscale factor (ablation)	0.5
LoRA r / α / dropout	16 / 32 / 0.05
InfoNCE temperature τ	0.05

Table 6: All pipeline hyperparameters (§3.1–§3.3). Each is a single documented, centralized value, not swept; see §6.

C Compute budget for contrastive fine-tuning

`lora_finetune.py` embeds each batch’s questions and tile images via `embed_questions/embed_tiles`, which support two paths: a padded, batched forward pass (`batched_embeddings=True`, the default) or one forward call per question/image (`--no-batched-embeddings`, a fallback if `scripts/verify_batching.py` – which cross-checks the batched path against a per-item reference before any real run – fails on a given environment). At batch size 4, $n=2$ hard negatives, 3 epochs over the 1,339 examples of Table 5, one optimizer step embeds 4 questions and 12 tile images; the unbatched fallback costs 16 sequential forward passes per step (1,005 steps $\times$ 16 = 16,080 total), all contributing to one shared backward. At an assumed 150–400 ms per forward call, that is 40–107 minutes of forward

compute alone, plausibly an hour to several hours wall-clock once backward and per-call overhead are included – an estimate, not a measurement, for a path we did not end up using.

The batched default instead issues one forward call for all 4 questions and one for all 12 tile images per step – far fewer Python-level calls, but each now holds every image’s activations in memory jointly rather than releasing them between items, and the vision encoder can still emit a large token count for our largest tiles (2048×2048 px). This traded fewer calls for a much larger peak footprint than the unbatched estimate above implied: a first attempt at the run reported in §5 (ablation (b)) hit `torch.OutOfMemoryError` on a routine dropout call, with 79.16 of 79.25 GiB already in use on an 80 GB A100/H100-class card – not fragmentation, genuine peak usage near the card’s full capacity. `--gradient-checkpointing` (recomputing activations on the backward pass instead of retaining them, at a $\sim 20\text{--}30\%$ slowdown) resolved it; with it, training completed 3 epochs without further incident. The practical takeaway: the batched path is faster per call but not simply "cheaper" in memory than the unbatched one, and `--gradient-checkpointing` should be treated as the default starting point on an 80 GB card at this tile resolution, not a fallback for smaller GPUs only. Wall-clock time for the completed run was not logged precisely enough to report a measured figure here.

D Tiling ablation: cost detail

	Tiles/page	Px/tile	Total px/page
DOM	24.0	157,170	3,772,076
Grid	4.9	2,057,956	10,061,116

Table 7: Tiling cost, same 18 pages as Table 2. DOM-guided tiling produces more, smaller tiles, yet a smaller *total* pixel budget per page ($2.7\times$ less) – a direct consequence of the text-flow bounding-box measurement of §3.1, which crops text tiles to the width their content actually occupies instead of the full page width every grid band uses regardless of content. If image tokens scale with pixel area, as they do for Qwen2-VL’s vision encoder, DOM-guided tiling need not trade accuracy for token cost against a blind grid.

Attach rate (the fraction of questions whose answer string survives a tiling’s 200-character text preview) is 51.8% for DOM (212/409) against 26.4% for grid (108/409) – consistent with grid

bands aggregating far more raw content into the same fixed text budget, diluting the specific snippet a lexical scorer needs to surface.

E Additional figures

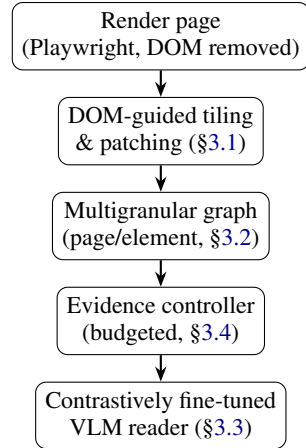


Figure 2: Pipeline overview. Rendering and tiling happen once per page; the evidence controller runs once per query, deciding which graph nodes the reader opens.

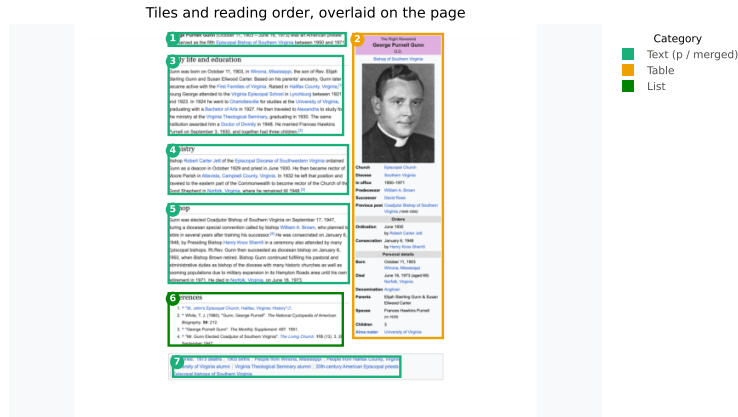


Figure 3: DOM-guided tiles (colored by content category, numbered in reading order) overlaid on the rendered page for `en.wikipedia.org/wiki/George_P._Gunn`. Tile boundaries follow the DOM’s own element boundaries: the infobox table (tile 2) is never split mid-row and stays disjoint from the body text and reference list that visually share its column range (tiles 1, 3–6), each section heading is paired with its first content block – a paragraph for tiles 3–5, the reference list itself for tile 6 – rather than emitted alone, and the trailing category footer (tile 7) is merged into its own tile rather than left as a flood of near-empty fragments.

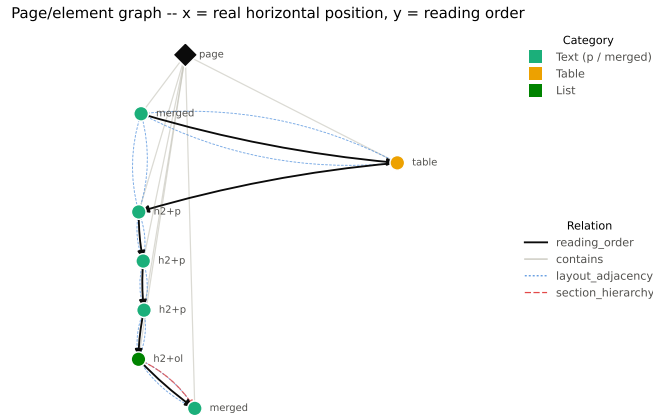


Figure 4: Page/element graph for the same page as Figure 3, laid out by real horizontal position (x) and reading-order rank (y). `reading_order` (solid black) and `contains` (light gray) are the two relations MAGE-RAG also has; `layout_adjacency` (dotted blue) and `section_hierarchy` (dashed red) are the two we add. Most headings here are paired with their content into one tile (§3.1), so `section_hierarchy` shows only where trailing non-heading content (the category footer) attaches to the last open section, rather than between the (sibling, not nested) section headings themselves.

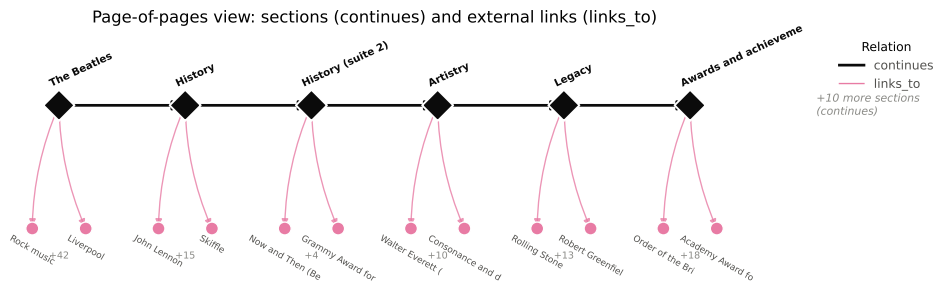


Figure 5: Page-level view of the graph for `en.wikipedia.org/wiki/The_Beatles` (278 DOM elements, split into 16 sections at `h1/h2` boundaries – only the first 6 are shown for legibility). Each diamond is a page node, one per section; `continues` edges (black) chain them in document order; `links_to` edges (pink) reach external pages, collapsed here from their true element-level source (§3.2) to stay at page granularity. The leftmost node is the entry point.

```
<page url="..." title="...">
  <element id="tile_0000" tag="merged"
    state="inactive">...</element>
  <element id="tile_0003" tag="p"
    state="opened">Gunn was born
    on October 11, 1903...</element>
  <element id="tile_0004" tag="h2+p"
    state="inactive">...</element>
  ...
</page>
```

Figure 6: Excerpt of the XML rendering of a page graph after the evidence controller has run on the query “When and where was George Purnell Gunn born?”. Only the paragraph tile carrying the birth date/place has been opened; the rest remains inactive or pruned.